Run your spec, and look at the failure message: `expected: >= 50000 / got: 0`. It's pretty terse and doesn't really communicate the intent of the code.

Update the expectation to describe how the code should behave, rather than what the value of a variable is.

About Our Company

We also want to check that our class is correctly storing all the information investors will want to know about the company. Add the following example inside the example group from the previous exercise, just after the other example:

11-matchers-included-in-rspec-expectations/exercises/public_company_spec.rb

```
it 'provides attributes' do
  expect(company.name).to eq('Nil')
  expect(company.value_per_share).to eq(10)
  expect(company.share_count).to eq(10_000)
  expect(company.market_cap).to eq(1_000_000)
end
```

When you run this new example, RSpec stops the test at the first failure, on `company.name`. We don't get to see whether or not any of the other attributes were correct.

Use a different matcher here that checks *all* the attributes, and reports on any differences between what we're expecting and how the code actually behaves.

Tokenizing Words

For this exercise, we're testing a *tokenizer* that breaks text into individual words. Add the following spec to a new file:

11-matchers-included-in-rspec-expectations/exercises/tokenizer_spec.rb

```
RSpec.describe Tokenizer do
  let(:text) do
    <<-EOS
      I am Sam.
      Sam I am.
      Do you like green eggs and ham?
    EOS
  end

  it 'tokenizes multiple lines of text' do
    tokenized = Tokenizer.tokenize(text)
    expect(tokenized.first(6)).to eq ['I', 'am', 'Sam.', 'Sam', 'I', 'am']
  end
end
```

Add the following incorrect implementation of the `Tokenizer` class to the top of your new file: